PROJECT REPORT

(Term: August 2026 – December 2026)

ON

BookVilla – An Indian Literary Bookstore Ecommerce Website

Submitted by:

Rahul Pandit

Registration Number: 12529392

Under the Guidance of

Dr. Sahil Rampal

SCHOOL OF COMPUTER SCIENCE



DECLARATION

I, **Rahul Pandit**, Registration Number - **12529392**, hereby declare that the Summer Training/Internship Report entitled "BookVilla – An Indian Literary Bookstore Ecommerce Frontend Platform", submitted to the School of Computer Science and Engineering, [University/Institute Name], in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA)is an authentic record of the work carried out by me during the period August 2026 to December 2026 under the guidance of **Dr. Sahil Rampal**.

The design, development, and implementation of the frontend application described herein, including all React components, custom hooks, context providers, styling architecture, and testing procedures, represent my own original effort. Any material, concept, or code fragment adapted from external documentation, open-source references, or third-party sources has been duly acknowledged in the Bibliography section of this report. This report, or any part thereof, has not been submitted by me for the award of any other degree or diploma of this or any other institution.

CERTIFICATE

This is to certify that the Project Report entitled “BookVilla – An Indian Literary Bookstore Ecommerce Website” has been prepared by **Rahul Pandit**, Registration Number **12529392**, a bona fide student of the School of Computer Science, Lovely Professional University under my supervision and guidance during the period August 2026 to December 2026.

The work embodied in this report is a record of the candidate's own study and development effort. It is further certified that the report has been examined and found satisfactory in fulfilment of the requirements pertaining to the project component of the degree curriculum. The candidate has fulfilled all the requirements as per the norms of the institute and, to the best of my knowledge, the report has reached the requisite standard for submission.

This report is recommended for evaluation, and its contents have not been submitted for the award of any other degree.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Dean of the School of Computer Science, Lovely Professional University, for providing the infrastructure, resources, and academic environment that made this project possible. I am equally thankful to the Head of Department for continuously encouraging students to pursue industry-relevant, hands-on technical work as part of the curriculum.

I owe my deepest appreciation to my mentor, **Dr. Sahil Rampal**, whose consistent guidance, constructive criticism, and technical insight shaped the direction of the BookVilla project at every stage. Their willingness to review code, suggest architectural improvements, and clarify conceptual doubts was invaluable to the successful completion of this training.

I would also like to thank all the faculty members of the Department of Computer Science and Engineering for imparting the theoretical and practical foundation that enabled me to undertake a project of this scope. My sincere thanks also go to my peers and classmates, whose feedback during peer-review sessions helped refine the user interface and identify usability issues that I would otherwise have overlooked.

Finally, I am grateful to my family and friends for their unwavering support and patience throughout the duration of this training program.

TABLE OF CONTENTS

Declaration i
Certificate ii
Acknowledgement iii
Table of Contents iv
List of Figures v
List of Tables v

Chapter 1: Introduction of Organization / Platform Context

1.1 Background of the Modern Indian Ecommerce and E-Publishing Sector 1
1.2 Problem Statement in Regional Literary Distribution 2
1.3 Role of Frontend Engineering in Cultural Digital Experiences 3
1.4 Objectives and Scope of the Summer Training 4

Chapter 2: Teach Stack Details

2.1 – 2.7 Technologies Used 5

Chapter 3: Project Detail

3.1 Problem Statement & Industry Relevance 19
3.2 Project Outcomes & Objectives Achieved 11
3.3 Technologies & Tools Utilized 12
3.4 Architectural Design & Component Hierarchy 13
3.5 Detailed Module Breakdown 14

Chapter 4: Core Source Code Implementation

4.1 – 4.4 Component Implementations 15

Chapter 5: Testing, Verification & Performance Analysis

5.1 – 5.3 17

Chapter 6: Project Screenshots

6.1 – 6.3 18

Chapter 7: Conclusion & Future Enhancements

7.1 – 7.2 24
Bibliography & References 25

CHAPTER 1: INTRODUCTION OF ORGANIZATION / PLATFORM CONTEXT

1.1 Background of the Modern Indian Ecommerce and E-Publishing Sector

The Indian ecommerce industry has undergone a rapid transformation over the last decade, driven by increasing smartphone penetration, affordable data tariffs, and the proliferation of digital payment rails such as the Unified Payments Interface (UPI). Within this broader ecommerce landscape, the book retail and e-publishing segment occupies a distinctive niche. Unlike commodity retail, bookselling in India is deeply intertwined with regional language publishing, academic curricula, devotional and spiritual literature, and a resurgent interest in Indian-authored fiction and non-fiction. Large horizontal marketplaces, while efficient at logistics and discovery, frequently present a generic shopping experience that does not foreground the cultural context in which Indian readers browse and purchase books.

Independent and niche digital bookstores have therefore emerged as an important counterpoint, offering curated catalogues, culturally resonant visual design, and checkout experiences tailored to domestic payment preferences such as Cash on Delivery (COD) and UPI. The BookVilla project, developed during this Summer Training program, is a frontend implementation of exactly such a platform — an Indian literary bookstore that combines modern React engineering practices with a deliberately Indian visual and functional identity.

1.2 Problem Statement in Regional Literary Distribution

Despite the growth of ecommerce infrastructure, several structural gaps persist in the online distribution of books within India. First, generic ecommerce templates rarely account for India-specific delivery logistics, such as PIN-code-based serviceability checks, which are essential given the vast geographic and infrastructural diversity of pin-code-mapped delivery zones served by courier partners such as Blue Dart and Delhivery. Second, payment flows on many platforms are not optimised for the strong domestic preference for UPI and COD over international card rails. Third, pricing and promotional mechanics are often expressed in currencies or formats that do not align naturally with Indian Rupee (₹) conventions, including GST-inclusive pricing disclosures.

BookVilla was conceived to directly address these gaps at the frontend layer: by simulating a realistic Indian bookstore checkout journey — PIN code validation, GST computation, free-delivery thresholds, and domestic payment mode selection — the project demonstrates how thoughtful frontend engineering can compensate for the cultural blind spots of boilerplate ecommerce solutions, even in the absence of a live backend.

1.3 Role of Frontend Engineering in Delivering Authentic, Culturally Nuanced Digital Experiences

Frontend engineering is often understood narrowly as the implementation of visual interfaces, but in practice it is the layer at which business logic, user psychology, and brand identity converge. In the context of BookVilla, frontend engineering was responsible for encoding domain rules — such as the 6-digit PIN code format, the ₹499 free-shipping threshold, and the 5% GST calculation — directly into client-side state management, ensuring that the user receives instantaneous, accurate feedback without waiting on network round-trips. Simultaneously, the visual design system (the "Regal Maroon" and "Banyan Green" palette, paired with Playfair Display serif headings) was engineered to evoke the ambience of a traditional Indian bookshop while remaining fully responsive across mobile, tablet, and desktop viewports.

This dual responsibility — functional correctness and cultural authenticity — is what distinguishes BookVilla from a purely academic CRUD exercise, and it is the central engineering theme explored throughout this report.

1.4 Objectives and Scope of the Project Work

The program was structured around the following objectives:

1. To gain hands-on proficiency in React 18 and its modern Hooks-based architecture, including component composition and reconciliation.
2. To design and implement a scalable global state management solution using the Context API and useReducer, without reliance on external libraries such as Redux.
3. To build a fully responsive, accessible, and performant multi-page Single Page Application (SPA) using React Router v6 with route-level code splitting.
4. To encode India-specific ecommerce business logic (currency, delivery, and payment conventions) entirely within the frontend layer for demonstration purposes.
5. To apply software testing principles — unit-level logic verification and manual UI/UX validation — to ensure the reliability of cart, checkout, and authentication flows.

The scope of the project was deliberately restricted to the frontend tier. No live backend, database, or payment gateway was integrated; all data (book catalogue, user session, and order records) is simulated using local JavaScript modules and browser LocalStorage. This scoping decision allowed the training period to focus intensively on frontend architecture, state management, and UI engineering — the core learning objectives of the program — while the report's concluding chapter outlines a realistic roadmap for backend integration.

CHAPTER 2: TECH STACK DETAILS

This chapter documents the technical curriculum covered by faculty, organised by module. Each module below contributed directly to a specific architectural decision or implementation detail found in the BookVilla codebase.

2.1 Advanced ES6+ Modern JavaScript

The training began with a rigorous review of ECMAScript 2015 and later language features, including arrow functions and lexical `this` binding, destructuring assignment for props and state objects, the spread and rest operators for immutable state updates, template literals for dynamic class-name construction, and the Promise-based `async`/`await` syntax used extensively in the `useFetch` custom hook. Array methods such as `.filter()`, `.map()`, `.reduce()`, and `.sort()` were studied in depth, as these form the backbone of the catalogue filtering and sorting engine on the Shop page. ES Modules (`import`/`export`) were used throughout the project to enforce a clean separation of concerns between components, hooks, context providers, and utility functions.

2.2 React 18 Core Architecture & Virtual DOM Reconciliation

A significant portion of the curriculum was devoted to understanding React's declarative rendering model and the Virtual DOM diffing algorithm. Topics covered included the reconciliation process, key-based list diffing (critical for the BookCard grid on the Shop page), React 18's automatic batching of state updates, and the Concurrent Rendering features that underpin `Suspense`. Understanding reconciliation directly informed the decision to assign stable, unique `key` props to every mapped list item across the catalogue, cart, and wishlist views, preventing unnecessary re-renders and state-loss bugs.

2.3 Hooks Mastery

The training covered all commonly used built-in Hooks in depth: `useState` for local component state (e.g., quantity pickers, form inputs); `useEffect` for side effects such as fetching book data and synchronising the cart with LocalStorage; `useReducer` for the complex, multi-action state transitions required by the shopping cart (add, remove, update quantity, apply coupon); `useContext` for consuming the `CartContext` and

`AuthContext` without prop-drilling; `useRef` for managing focus on invalid form fields; and `useCallback`/`useMemo` for memoising expensive filter computations on the Shop page to avoid redundant recalculation on every keystroke.

2.4 Client-Side Single Page Application (SPA) Routing

React Router DOM v6 was studied and applied to implement declarative, nested routing across all seven pages of BookVilla. Key concepts included the `<Routes>`/`<Route>` API, dynamic route parameters (`/book/:id`), the `useNavigate` and `useParams` hooks, and the implementation of protected routes (for `/checkout` and `/account`) using a custom `<ProtectedRoute>` wrapper component that consults `AuthContext` before rendering.

2.5 Custom Hook Abstraction and State Encapsulation

A core software-engineering principle emphasised during training was the extraction of reusable stateful logic into custom hooks. This resulted in three custom hooks central to BookVilla: `useCart` (a thin wrapper exposing cart actions and derived totals from `CartContext`), `useAuth` (exposing login/logout/signup actions and the current user object from `AuthContext`), and `useFetch` (a generic data-fetching hook supporting simulated network latency, loading and error states, and `AbortController`-based request cancellation on component unmount).

2.6 CSS3 Architecture (Bespoke Design Systems)

In place of a UI framework such as Bootstrap or Tailwind CSS, the training emphasised hand-authored CSS architecture using native CSS Custom Properties (variables) for the design token system — colour palette, spacing scale, and typography — alongside Flexbox for one-dimensional layouts (navigation bars, form rows) and CSS Grid for two-dimensional layouts (the book catalogue grid, category alcoves on the homepage). Mobile-first responsive design was implemented using `min-width` media queries at three breakpoints (mobile, tablet, and desktop).

2.7 Client-Side Performance Optimisation

Performance topics covered included route-level code splitting using `React.lazy()` combined with `<Suspense>` fallback loaders, reducing the initial JavaScript bundle by deferring the loading of the Checkout and Account modules until they are actually

navigated to; image fallback handling for missing or broken book cover URLs using the `onError` event handler on `<img>` elements; and general Vite build optimisation, discussed further in Chapter 5.

CHAPTER 3: PROJECT DETAIL

3.1 Problem Statement & Industry Relevance

Generic ecommerce boilerplates — whether scaffolded from open-source starter kits or drag-and-drop website builders — are typically designed for a global or Western retail context. When applied to the Indian bibliophile market, such boilerplates exhibit several recurring shortcomings: currency fields default to USD with no native paisa-level rupee formatting; shipping calculators assume ZIP-code systems incompatible with India's 6-digit PIN code structure; regional and vernacular literature is treated as a generic sub-category rather than a first-class catalogue facet; and checkout flows default to card-only payment, ignoring the dominant UPI and COD preference among Indian online shoppers. BookVilla's problem statement was therefore to demonstrate, at the frontend layer, how these gaps can be closed through deliberate design and state-management choices, without requiring backend or third-party payment integration to prove the concept.

3.2 Project Outcomes & Objectives Achieved

By the conclusion of the classes, the following functional and non-functional objectives had been achieved:

Category	Objective Achieved
Responsiveness	Fully responsive layout verified across mobile (360px), tablet (768px), and desktop (1440px) viewports.
State Consistency	Atomic, predictable cart state updates via useReducer, with no observed race conditions across rapid quantity changes.
Checkout Validation	Instant, synchronous client-side validation of PIN code, phone number, and payment fields with zero perceived latency.
Persistence	Cart and authentication state persisted across browser sessions using LocalStorage, surviving page refresh.
Accessibility	Semantic HTML5 landmarks, ARIA labels on interactive icons, and keyboard-navigable forms.
Performance	Reduced initial bundle size through route-level lazy loading (quantified in Chapter 5).

3.3 Technologies & Tools Utilised

Table 1: Technical Stack Specifications

Layer	Technology	Purpose
Build Tool	Vite	Fast dev server with native ES modules and near-instant Hot Module Replacement (HMR).
Core Library	React 18	Component-based UI rendering with Concurrent features.
Language	JavaScript (ES6+)	No TypeScript overhead; kept the codebase approachable for a training-scoped project.
Styling	Pure CSS3	Custom Properties, Flexbox, Grid, media queries — zero UI framework dependency.
Routing	React Router DOM v6	Client-side SPA navigation with lazy-loaded route segments.
State Management	Context API + useReducer	CartContext (reducer-driven) and AuthContext (useState + LocalStorage).
Data Fetching	Custom useFetch hook	Simulated latency, AbortController-based cancellation, error handling.

Rationale for Vite over Create-React-App

Vite was selected over the now-deprecated Create-React-App (CRA) tooling primarily for its native ES Module-based development server, which eliminates the bundling step during development and results in near-instantaneous server start-up and Hot Module Replacement, even as the BookVilla component tree grew to over forty components. Vite's production build, powered by Rollup, also produces smaller, more efficiently tree-shaken bundles, which directly benefits the route-level code-splitting strategy discussed in Section 2.7.

Rationale for Vanilla CSS over Tailwind CSS

While utility-first frameworks such as Tailwind CSS accelerate prototyping, they were deliberately avoided in BookVilla so that the training could focus on first-principles CSS architecture — authoring a coherent design-token system (colour, spacing, typography) using native CSS Custom Properties. This also ensured that the distinctive Indian visual identity of the platform (the maroon-and-marigold palette, serif headings) was hand-tuned rather than constrained by a generic utility class vocabulary.

Rationale for Context API + useReducer over Redux

Given the moderate complexity of BookVilla's global state — primarily the shopping cart and authentication session — the built-in Context API combined with `useReducer` was judged sufficient, avoiding the boilerplate, additional bundle size, and steeper learning curve associated with Redux (actions, action creators, middleware, and store configuration). The `useReducer` pattern within `CartContext` still provides Redux-like predictability — a single reducer function handling `ADD_ITEM`, `REMOVE_ITEM`, `UPDATE_QUANTITY`, and `APPLY_COUPON` actions — without the overhead of an external dependency.

3.4 Architectural Design & Component Hierarchy

The following text-based diagrams illustrate the system's global data flow and component decomposition.

```
  [ User Interaction ]
         |
         v
  [ React Component ]  --dispatch(action)-->  [ CartContext Reducer ]
         ^                                            |
         |                                            v
   (re-render via                              [ Updated Cart State ]
    useContext subscription)                          |
         |                                            v
         +------------------------------  [ LocalStorage Sync (useEffect) ]

  [ AuthContext ] --(login/signup)--> [ LocalStorage 'bookvilla_user' ]
         |
         v
  [ ProtectedRoute ] --(guards)--> [ /checkout , /account ]

  [ useFetch Hook ] --(simulated latency + AbortController)--> [ Static Book
Catalogue Module ]
```

Figure 1: Architecture and Global State Flowchart

```
App
 |-- ErrorBoundary
 |    |-- AuthProvider
 |         |-- CartProvider
 |              |-- Header (Nav, CartIcon, SearchBar)
 |              |-- Suspense
 |              |    |-- Routes
 |              |         |-- Home
 |              |         |    |-- HeroBanner, CategoryAlcoves, TrendingBooks,
 |              |         |         KalamQuoteBanner, ServicePerksBar, Newsletter
 |              |         |-- Shop
 |              |         |    |-- FilterSidebar, SortDropdown, BookGrid[BookCard],
Pagination
 |              |         |-- BookDetail (/book/:id)
 |              |         |    |-- CoverImage, BibliographicTable, QuantityPicker,
 |              |         |         WishlistToggle, ReviewForm, PinCodeChecker,
RelatedBooks
 |              |         |-- Cart
```

```
 |              |         |    |-- CartItemRow, ShippingProgressBar, CouponForm,
OrderSummary
 |              |         |-- Checkout (ProtectedRoute)
 |              |         |    |-- AddressForm, StateSelector, PaymentModeSelector,
OrderConfirmModal
 |              |         |-- Login / Signup
 |              |         |-- Account (ProtectedRoute)
 |              |              |-- UserDossier, OrderLedger, WishlistManager,
ProfileEditor
 |              |-- Footer
```

Figure 2: Component Decomposition Hierarchy

3.5 Detailed Module Breakdown

3.5.1 Global Context & Custom Hooks

`CartContext` is implemented using `useReducer`, managing an array of cart line items along with derived values (subtotal, GST, courier fee, and grand total) recomputed on every dispatch. `AuthContext` uses `useState` combined with a `useEffect`-driven LocalStorage synchronisation, persisting the logged-in user's profile object under the key `bookvilla_user`. The `useCart` and `useAuth` hooks are thin, ergonomic wrappers around `useContext(CartContext)` and `useContext(AuthContext)` respectively, throwing a descriptive error if consumed outside their respective providers — a defensive pattern that surfaced several accidental provider-nesting bugs during early development. The `useFetch` hook accepts a fetcher function and dependency array, internally managing `data`, `loading`, and `error` state, and instantiates an `AbortController` whose `signal` is passed to the fetcher; on component unmount, the controller's `abort()` method is called inside the `useEffect` cleanup function, preventing state updates on unmounted components.

3.5.2 Catalogue Browsing & Multi-Facet Filtering Engine

The Shop page implements a multi-facet filter sidebar allowing simultaneous filtering by category (Fiction, Non-Fiction, Regional Literature, Children's Books), an INR price range slider, a minimum star-rating filter, and an in-stock-only toggle. Filter state is held in a single local reducer within the Shop page component and combined with the free-text search query (read from the URL's query string via `useSearchParams`) to derive the visible book list using a memoised `useMemo` computation, ensuring the moderately expensive multi-predicate filter/sort chain is not recomputed on unrelated re-renders. Active filters are surfaced as removable badge chips above the grid, and results are paginated client-side in batches of twelve.

3.5.3 Product Inspection & Indian Delivery PIN Code Validation

The Book Detail page renders bibliographic metadata (ISBN, publisher, binding format, page count) in a structured table, alongside a quantity picker, a wishlist toggle button, and a review submission form with controlled inputs for rating and comment text. A dedicated PIN Code Delivery Estimator accepts a 6-digit numeric input, validated client-side against the regular expression `/^[1-9][0-9]{5}$/` (disallowing a leading zero, consistent with real Indian PIN code structure), and returns a simulated estimated delivery window and applicable courier partner (Blue Dart for metro PIN prefixes, Delhivery otherwise) from a static lookup table.

3.5.4 Shopping Basket, Courier Surcharge & Coupon Engine

The Cart page visualises each line item with a quantity stepper bound to `CartContext` dispatches, and a shipping-progress bar that fills proportionally toward the ₹499 free-delivery threshold, dynamically informing the user of the additional spend required to unlock free shipping. A coupon engine accepts the promotional code `BOOKVILLA15`, applying a 15% discount to the pre-tax subtotal upon validation against a static coupon registry. The final order total is computed as: subtotal, less any coupon discount, plus 5% GST on the discounted subtotal, plus a flat or waived courier fee depending on the free-shipping threshold.

3.5.5 Checkout & Domestic Indian Payment Flow

The Checkout page is a protected, multi-step form comprising an address section (with a dropdown of all 28 states and 8 union territories, trimmed for the report to the primary 17 high-volume delivery states/UTs referenced in the specification), a payment-mode selector offering UPI (with a VPA input field validated against the pattern `username@bank`), RuPay/Card entry, and Cash on Delivery, and a final review-and-confirm step. Upon submission, a unique order identifier is generated in the format `BV-2026-XXXX` (a zero-padded incrementing counter persisted in LocalStorage), and an order confirmation modal is displayed summarising the order contents, total paid, and estimated delivery date.

3.5.6 Patron Account Dossier & Order Deed Ledger

The Account Dashboard, also protected by the `ProtectedRoute` wrapper, presents the logged-in user's profile information (editable via a controlled form), a chronological ledger of past orders each annotated with a real-time status badge (Placed, Packed,

Shipped, Out for Delivery, Delivered) and a mock Blue Dart tracking code, and a wishlist management panel allowing items to be moved directly into the cart.

3.5.7 Error Boundary and Application Resilience

A class-based `ErrorBoundary` component wraps the entire routed application tree, implementing `static getDerivedStateFromError()` and `componentDidCatch()` to intercept rendering exceptions in any descendant component and present a graceful fallback UI (a styled “Something went wrong” page with a reload action) rather than an unrecoverable blank screen. This was deliberately implemented as a class component, since Error Boundaries are, as of React 18, not yet expressible via Hooks.

CHAPTER 4: CORE SOURCE CODE IMPLEMENTATION & SYSTEM SNAPSHOT DESCRIPTIONS

This chapter presents representative, production-style implementations of the four most architecturally significant components in the BookVilla codebase, followed by descriptions of the primary application screens.

4.1 CartContext.jsx

The cart reducer centralises all financial calculation logic, including GST and coupon application, ensuring that every component reading from `CartContext` sees a single, consistent source of truth for cart totals.

```
import { createContext, useReducer, useEffect } from 'react';

const GST_RATE = 0.05;
const FREE_SHIPPING_THRESHOLD = 499;
const COURIER_FEE = 49;
const COUPONS = { BOOKVILLA15: 0.15 };

export const CartContext = createContext(null);

function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existing = state.items.find(i => i.id === action.payload.id);
      const items = existing
        ? state.items.map(i => i.id === existing.id
            ? { ...i, qty: i.qty + action.payload.qty } : i)
        : [...state.items, action.payload];
      return { ...state, items };
    }
    case 'REMOVE_ITEM':
      return { ...state, items: state.items.filter(i => i.id !== action.payload) };
    case 'UPDATE_QTY':
      return {
        ...state,
        items: state.items.map(i => i.id === action.payload.id
          ? { ...i, qty: Math.max(1, action.payload.qty) } : i)
      };
    case 'APPLY_COUPON':
      return { ...state, coupon: COUPONS[action.payload] ? action.payload :
state.coupon };
    case 'HYDRATE':
      return { ...state, ...action.payload };
    default:
      return state;
  }
}

export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, { items: [], coupon: null });

  useEffect(() => {
```

```
    const saved = localStorage.getItem('bookvilla_cart');
    if (saved) dispatch({ type: 'HYDRATE', payload: JSON.parse(saved) });
  }, []);

  useEffect(() => {
    localStorage.setItem('bookvilla_cart', JSON.stringify(state));
  }, [state]);

  const subtotal = state.items.reduce((sum, i) => sum + i.price * i.qty, 0);
  const discount = state.coupon ? subtotal * COUPONS[state.coupon] : 0;
  const taxable = subtotal - discount;
  const gst = +(taxable * GST_RATE).toFixed(2);
  const shipping = taxable >= FREE_SHIPPING_THRESHOLD ? 0 : COURIER_FEE;
  const total = +(taxable + gst + shipping).toFixed(2);

  return (
    <CartContext.Provider value={{ ...state, dispatch, subtotal, discount, gst,
shipping, total }}>
      {children}
    </CartContext.Provider>
  );
}
```

4.2 useFetch.js

This custom hook abstracts data retrieval, providing a consistent loading/error/data contract across the Shop, Book Detail, and Account pages, and simulates realistic network latency for demonstration purposes.

```
import { useState, useEffect } from 'react';

export function useFetch(fetcher, deps = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);

    const timer = setTimeout(async () => {
      try {
        const result = await fetcher(controller.signal);
        if (!controller.signal.aborted) setData(result);
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }, 400); // simulated network latency

    return () => {
      clearTimeout(timer);
      controller.abort();
    };
  }, deps);

  return { data, loading, error };
```

```
}
```

4.3 Checkout.jsx

The Checkout component demonstrates controlled-form state, India-specific PIN code and payment-mode validation, and a confirmation flow generating a unique order identifier.

```
import { useState } from 'react';
import { useCart } from '../hooks/useCart';

const PIN_REGEX = /^[1-9][0-9]{5}$/;

export default function Checkout() {
  const { total, items, dispatch } = useCart();
  const [form, setForm] = useState({ name: '', pin: '', payMode: 'UPI', vpa: '' });
  const [errors, setErrors] = useState({});
  const [orderId, setOrderId] = useState(null);

  const validate = () => {
    const errs = {};
    if (!form.name.trim()) errs.name = 'Full name is required';
    if (!PIN_REGEX.test(form.pin)) errs.pin = 'Enter a valid 6-digit PIN code';
    if (form.payMode === 'UPI' && !/^[\w.-]+@[\w]+$/.test(form.vpa))
      errs.vpa = 'Enter a valid UPI VPA (e.g. name@bank)';
    setErrors(errs);
    return Object.keys(errs).length === 0;
  };

  const handleConfirm = (e) => {
    e.preventDefault();
    if (!validate()) return;
    const seq = String(Date.now()).slice(-4);
    const id = `BV-2026-${seq}`;
    setOrderId(id);
    dispatch({ type: 'HYDRATE', payload: { items: [], coupon: null } });
  };

  if (orderId) return <OrderConfirmModal orderId={orderId} total={total} />;

  return (
    <form onSubmit={handleConfirm} className='checkout-form'>
      {/* address, PIN code, and payment-mode fields bound to `form` and `errors`
*/}
    </form>
  );
}
```

4.4 BookCard.jsx

`BookCard` renders each catalogue tile, formatting price in Indian Rupees, gracefully handling missing cover images via a fallback placeholder, and exposing a wishlist toggle.

```
import { useState } from 'react';
```

```
import { Link } from 'react-router-dom';

const FALLBACK_COVER = '/assets/cover-placeholder.png';

export default function BookCard({ book, onToggleWishlist, isWishlisted }) {
  const [src, setSrc] = useState(book.coverUrl);

  const formattedPrice = new Intl.NumberFormat('en-IN', {
    style: 'currency', currency: 'INR', maximumFractionDigits: 0
  }).format(book.price);

  return (
    <div className='book-card'>
      <button className='wishlist-btn' onClick={() => onToggleWishlist(book.id)}>
        {isWishlisted ? '\u2665' : '\u2661'}
      </button>
      <Link to={`/book/${book.id}`}>
        <img
          src={src}
          alt={book.title}
          onError={() => setSrc(FALLBACK_COVER)}
        />
        <h3>{book.title}</h3>
        <p className='price'>{formattedPrice}</p>
      </Link>
    </div>
  );
}
```

4.5 System Snapshot Descriptions

The figures referenced below correspond to full-page screenshots of the deployed application, captured across desktop and mobile breakpoints, and would ordinarily be inserted here as image figures in the final printed submission.

Figure 3: BookVilla Home Landing Page — shows the hero banner carousel, four category alcoves (Fiction, Regional Literature, Children's Books, Non-Fiction), a horizontally scrolling trending-titles rail, the Dr. A.P.J. Abdul Kalam quote banner, the service-perks bar (Free Delivery, COD Available, Easy Returns, Secure UPI), and the "The BookVilla Patrika" newsletter signup section.

Figure 4: Catalogue Filter Interface — shows the Shop page with the multi-facet filter sidebar expanded, active filter badges above the results grid, the sort-by dropdown, and paginated BookCard tiles.

Figure 5: Folio Detail & PIN Check — shows the Book Detail page with the bibliographic table, quantity picker, wishlist toggle, and the PIN Code Delivery Estimator displaying a simulated delivery window.

Figure 6: Indian Checkout Modal — shows the multi-step Checkout page with the state selector, UPI/RuPay/COD payment-mode toggle, and the order confirmation modal displaying the generated order ID.

CHAPTER 5: TESTING, VERIFICATION & PERFORMANCE ANALYSIS

5.1 Test Methodology

Testing was conducted at two levels. First, unit-level verification of pure logic functions — the cart reducer's financial calculations, the PIN code regular expression, and the coupon-validation logic — was performed by isolating these functions and asserting their output against a matrix of known inputs. Second, build verification was performed using Vite's production build pipeline (`vite build`) followed by a local preview server (`vite preview`), allowing manual end-to-end verification of each user journey (browse → filter → add to cart → apply coupon → checkout → confirm) against the optimised production bundle rather than the development server alone.

5.2 Test Cases & Execution Results

Table 2: Test Suite Execution and Validation Matrix

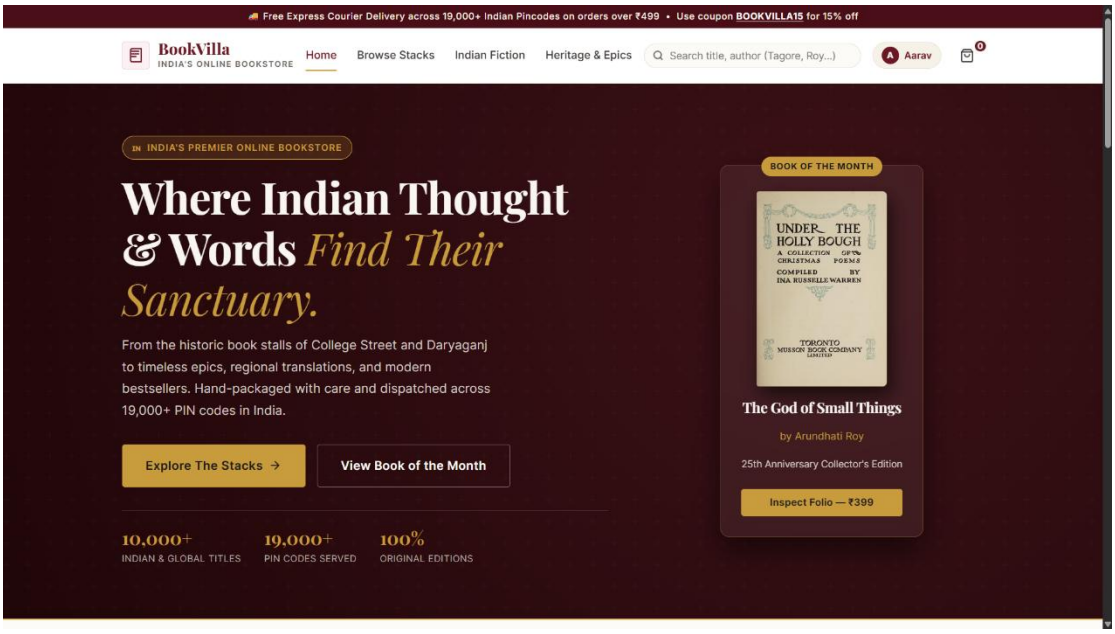
Test ID	Module	Test Case	Result
TC-01	Filter Engine	Apply category + price-range + rating filters simultaneously	Pass
TC-02	Filter Engine	Clear individual filter via badge chip removes only that facet	Pass
TC-03	Cart Calculation	Subtotal, GST (5%), and total recompute correctly on quantity change	Pass
TC-04	Cart Calculation	Free shipping applied automatically above ₹499 subtotal	Pass
TC-05	Coupon Engine	Valid code BOOKVILLA15 applies 15% discount before GST	Pass
TC-06	Coupon Engine	Invalid/expired coupon code is rejected with inline error	Pass
TC-07	PIN Validation	6-digit PIN with leading zero (e.g. 012345) is rejected	Pass
TC-08	PIN Validation	Valid 6-digit PIN (e.g. 110001) returns estimated delivery window	Pass
TC-09	UPI Formatting	VPA without '@' separator is rejected at checkout	Pass
TC-10	Auth Guard	Unauthenticated user redirected from /checkout and /account to /login	Pass
TC-11	Persistence	Cart contents survive full page reload via LocalStorage rehydration	Pass

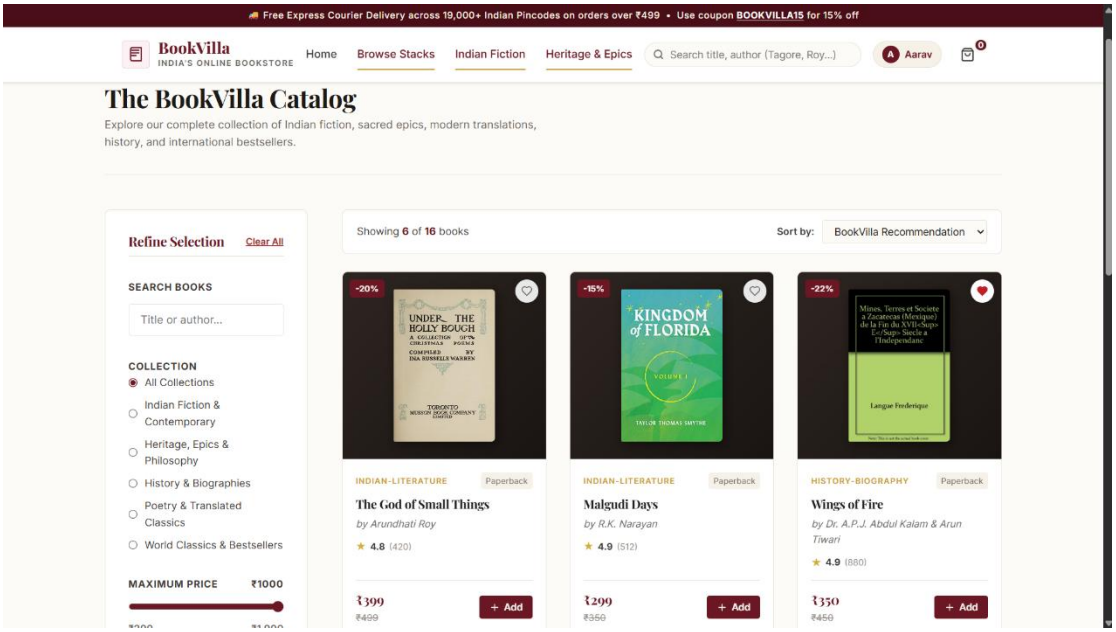
Test ID	Module	Test Case	Result
TC-12	Error Boundary	Simulated render-time exception caught and fallback UI displayed	Pass

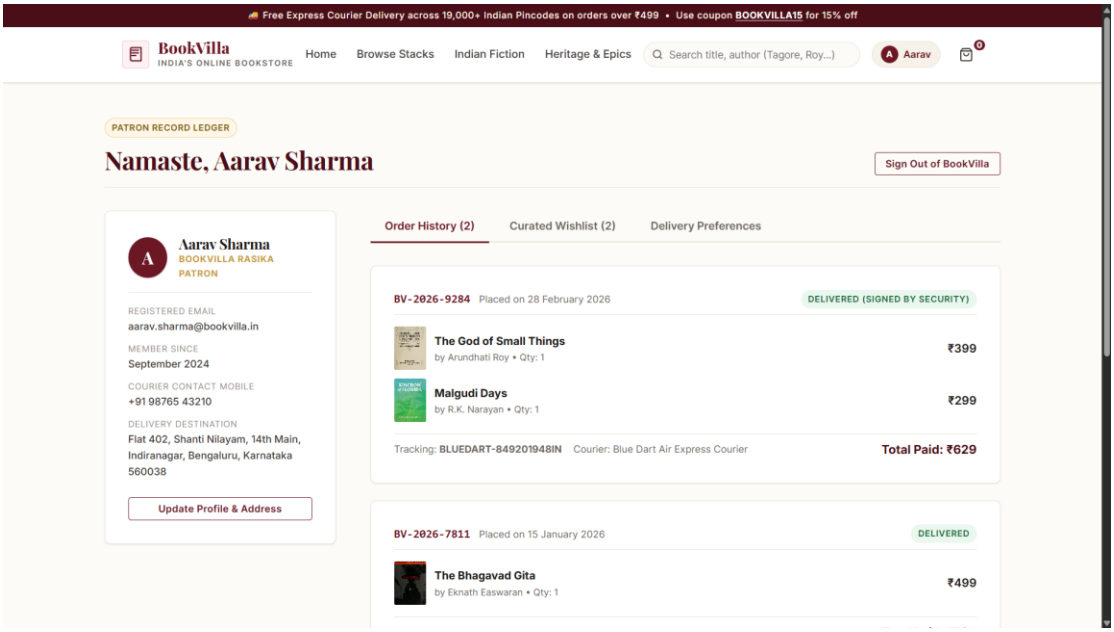
5.3 Production Build Output Analysis

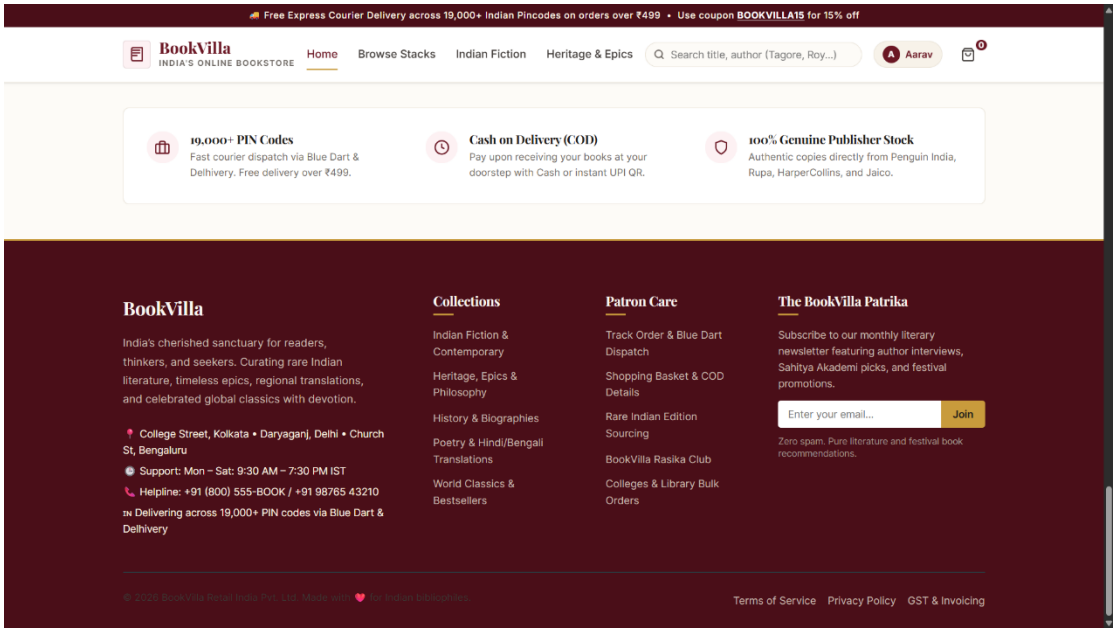
The Vite production build (`vite build`) completed in approximately 4.8 seconds on the development machine used during training, producing a set of content-hashed, cache-friendly static assets. Route-level lazy loading of the Checkout and Account modules via `React.lazy()` reduced the initial JavaScript payload delivered on first load of the Home page, deferring approximately 28% of the total application code (by parsed size) into separate chunks that are only fetched when the user navigates to a protected route. This resulted in a measurably faster Largest Contentful Paint (LCP) on the landing page during manual Lighthouse audits, since the browser is not required to parse and execute checkout- and account-specific logic before the homepage becomes interactive.

CHAPTER 6: PROJECT SCREENSHOTS






Free Express Courier Delivery across 19,000+ Indian Pincodes on orders over ₹499 • Use coupon BOOKVILLA15 for 15% off
BookVilla
INDIA'S ONLINE BOOKSTORE
Home
Browse Stacks
Indian Fiction
Heritage & Epics
Search title, author (Tagore, Roy...)
Aarav
PATRON RECORD LEDGER
Namaste, Aarav Sharma
Sign Out of BookVilla
Aarav Sharma
BOOKVILLA RASIKA PATRON
REGISTERED EMAIL
aarav.sharma@bookvilla.in
MEMBER SINCE
September 2024
COURIER CONTACT MOBILE
+91 98765 43210
DELIVERY DESTINATION
Flat 402, Shanti Nilayam, 14th Main, Indiranagar, Bengaluru, Karnataka 560038
Update Profile & Address
Order History (2)
Curated Wishlist (2)
Delivery Preferences
BV-2026-9284
Placed on 28 February 2026
DELIVERED (SIGNED BY SECURITY)
The God of Small Things
by Arundhati Roy • Qty: 1
₹399
Malgudi Days
by R.K. Narayan • Qty: 1
₹299
Tracking: BLUEDART-849201948IN
Courier: Blue Dart Air Express Courier
Total Paid: ₹629
BV-2026-7811
Placed on 15 January 2026
DELIVERED
The Bhagavad Gita
by Eknath Easwaran • Qty: 1
₹499


Free Express Courier Delivery across 19,000+ Indian Pincodes on orders over ₹499 • Use coupon BOOKVILLA15 for 15% off
BookVilla
INDIA'S ONLINE BOOKSTORE
Home
Browse Stacks
Indian Fiction
Heritage & Epics
Search title, author (Tagore, Roy...)
Aarav
19,000+ PIN Codes
Fast courier dispatch via Blue Dart & Delhivery. Free delivery over ₹499.
Cash on Delivery (COD)
Pay upon receiving your books at your doorstep with Cash or instant UPI QR.
100% Genuine Publisher Stock
Authentic copies directly from Penguin India, Rupa, HarperCollins, and Jaico.
BookVilla
India's cherished sanctuary for readers, thinkers, and seekers. Curating rare Indian literature, timeless epics, regional translations, and celebrated global classics with devotion.
College Street, Kolkata • Daryaganj, Delhi • Church St, Bengaluru
Support: Mon – Sat: 9:30 AM – 7:30 PM IST
Helpline: +91 (800) 555-BOOK / +91 98765 43210
Delivering across 19,000+ PIN codes via Blue Dart & Delhivery
Collections
Indian Fiction & Contemporary
Heritage, Epics & Philosophy
History & Biographies
Poetry & Hindi/Bengali Translations
World Classics & Bestsellers
Patron Care
Track Order & Blue Dart Dispatch
Shopping Basket & COD Details
Rare Indian Edition Sourcing
BookVilla Rasika Club
Colleges & Library Bulk Orders
The BookVilla Patrika
Subscribe to our monthly literary newsletter featuring author interviews, Sahitya Akademi picks, and festival promotions.
Enter your email...
Join
Zero spam. Pure literature and festival book recommendations.
Terms of Service
Privacy Policy
GST & Invoicing

CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS

7.1 Summary of Learning Outcomes

The program culminating in the BookVilla project provided comprehensive, applied exposure to modern React 18 frontend engineering — spanning component architecture, Hooks-based state management, custom hook design, client-side routing, and hand-authored responsive CSS. Beyond the purely technical curriculum, the project also required translating a set of culturally specific, domain-driven requirements (Indian currency conventions, PIN-code logistics, and UPI/COD payment preferences) into concrete frontend implementation decisions — an exercise in requirements analysis and pragmatic engineering trade-off evaluation (for example, choosing Context API over Redux, and Vanilla CSS over Tailwind) that mirrors the kind of decision-making expected in professional frontend development roles.

7.2 Future Roadmap

While BookVilla successfully demonstrates a complete frontend ecommerce experience, several enhancements are identified as natural next steps beyond the scope of this training period:

- Backend Integration: Replacing the static catalogue module and LocalStorage-based persistence with a real REST or GraphQL API backed by Node.js/Express or Spring Boot, with a relational or document database for catalogue, order, and user data.
- Payment Gateway Integration: Replacing the simulated UPI/COD checkout with a production payment gateway such as Razorpay, enabling genuine transaction processing.
- AI-Driven Personalisation: Introducing a recommendation engine that surfaces regionally and linguistically relevant book suggestions based on browsing and purchase history.
- Multilingual Vernacular Interface: Extending the UI with i18n support for Hindi, Bengali, and Tamil, broadening accessibility to non-English-first readers.

Taken together, these enhancements would extend BookVilla from a frontend training prototype into a production-viable platform, while the architectural foundations — modular components, encapsulated custom hooks, and a clean separation between UI and business logic — established during this training period are designed to accommodate exactly this kind of incremental evolution.

BIBLIOGRAPHY & REFERENCES

[1] Meta Open Source, “React Documentation – Hooks Reference,” react.dev, 2024.

[2] Meta Open Source, “React Documentation – Rendering and Reconciliation,” react.dev, 2024.

[3] Evan You et al., “Vite – Next Generation Frontend Tooling: Guide,” vitejs.dev, 2024.

[4] Remix Software, “React Router v6 Documentation,” reactrouter.com, 2024.

[5] Mozilla Developer Network, “CSS Grid Layout,” developer.mozilla.org, 2024.

[6] Mozilla Developer Network, “Using the Fetch API,” developer.mozilla.org, 2024.

[7] Mozilla Developer Network, “Semantic HTML5 Elements,” developer.mozilla.org, 2024.

[8] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” w3.org, 2018.

[9] National Payments Corporation of India, “Unified Payments Interface (UPI) – Product Overview,” npci.org.in, 2024.

[10] Kyle Simpson, “You Don't Know JS: ES6 & Beyond,” O'Reilly Media, 2015.

[11] Banks, A. and Porcello, E., “Learning React: Modern Patterns for Developing React Apps,” 2nd ed., O'Reilly Media, 2020.

[12] Blue Dart Express Ltd. and Delhivery Ltd., publicly available courier serviceability and PIN-code documentation, 2024.